

Modularity in C

Luca Didaci

Table of contents

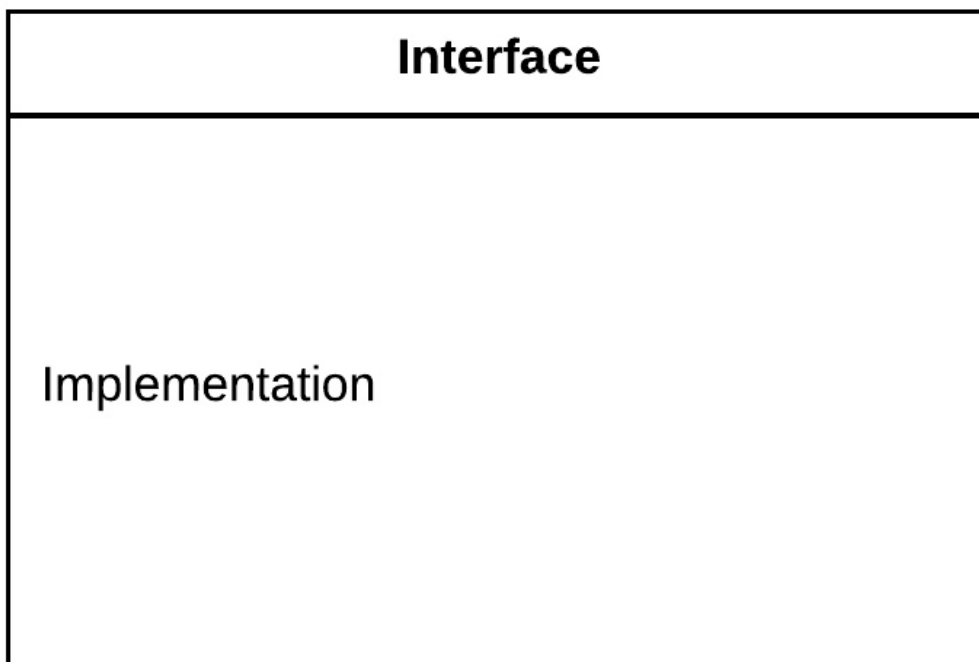
1	MODULARITY in C	1
1.1	Module: interface + implementation.	1
1.2	Who includes who?	4
1.3	Exercise	6
1.4	Solution	7
1.5	Exercise	9
1.6	Solution	10
1.7	Discussion about multiple declarations	11
1.8	Other details	12
2	Modularity in C: Encapsulation	15
2.1	Encapsulation: a possible solution	16
2.2	Implications	20
2.3	Managing the Data	20
2.4	Why Must Memory Be Allocated Dynamically?	20
2.5	Some notes on modularity	21
2.6	Un esempio di memory leakage	22
2.7	Soluzione	23
2.8	Discussione	24
2.9	Common errors	25
3	ESERCIZI	31

1 MODULARITY in C

1.1 Module: interface + implementation.

- Interface: **what a module does.** *Declares* identifiers, types, and routines that are available to the clients.
- Implementation: **specifies how the module functions internally to achieve its behavior..**

We can have one interface, many implementations



A module comes in two parts, its interface and its implementation. The interface specifies what a module does. It declares identifiers, types, and routines that are available to the clients. An implementation specifies how a module accomplishes the purpose advertised by its interface.

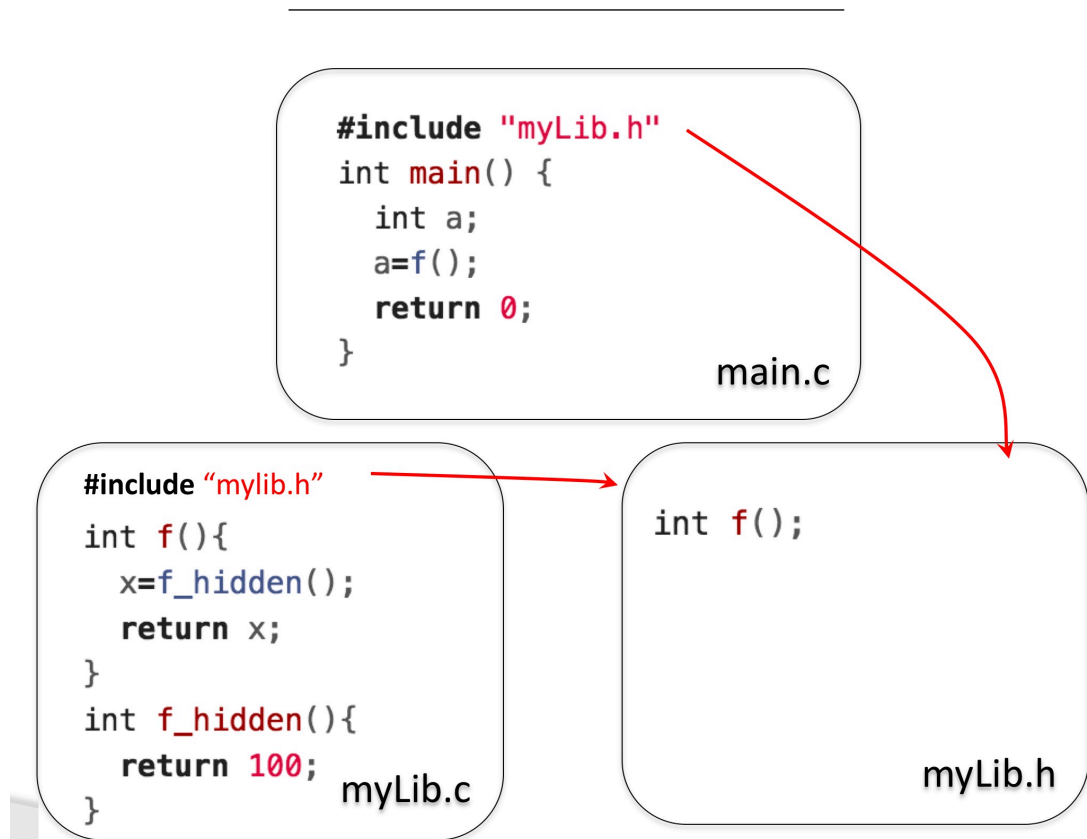
1 MODULARITY in C

For a given module, there is usually one interface, but there might be many implementations. Each implementation might use different algorithms and data structures, but they all must meet the specifications given by the interface. This methodology helps to avoid unnecessary code duplication and helps avoid bugs: interfaces and implementations are written and debugged once, but used often. An interface specifies only those identifiers that clients may use, hiding irrelevant representation details and algorithms. This helps clients avoid dependencies on a particular implementation.

Interface in C is specified by a **header file** (.h extension).

A client **imports** an interface with the C preprocessor **#include** directive.

The **#include** preprocessor directive causes the compiler to replace that line with the **entire text of the contents of the named file**.



1.1 Module: interface + implementation.

An interface is specified by a header file (`.h` extension). A client imports an interface with the C preprocessor `#include` directive.

An implementation is provided by one or more `.c` files. *Implementation includes its interface's .h file* to ensure that its definitions are consistent with the interface's declarations.

You request to use a header file in your program by **including it with the C preprocessing directive `#include`**.

Including a header file is equal to copying the content of the header file. We don't copy it manually. The manual copy is a time-consuming, error-prone action that is impossible to manage if we have multiple source files in a program.

*Implementation includes **its** interface (.h file) to ensure consistency.*

Header files that **declare** functions (or external variables) should be **included** in the file that *defines* the function or variable.

i.e. `myFile.c` must include `myFile.h`

In that way, the compiler can do **type checking**, and the external declaration will always agree with the definition.

The compiler checks if there is consistency between the data types declared in the interface and those used in the implementation.

This is the final result after the preprocessor includes `"mylib.h"` in `myLib.c`

```
int f(); # ---> FROM mylib.h

int f(){
    int x;
    x=f_hidden();
    return x;
}

int f_hidden(){
    return 100;
}
```

1.2 Who includes who?

The gold rule is: **#include .h files into .c files**

Avoid to **#include .h files** in other **.h files**.

Every header file **.h** should **#include** every other header file that it requires to compile correctly, **but no more**.

The reason is simple: if **file_1.h** includes **files_2.h** that includes **file_3.h** that includes **file_4.h...** when your **main.c** includes **file_1.h**, you are including an **uncontrollable chain of header files**.

If you include **.h files** into **.c files**, you have more control over what you include

YES - I make it explicit that module **FILE_1** uses modules **FILE_2** but not **FILE_3**.

```
// -----  
// file_1.c  
// -----  
  
#include "file_1.h"  
#include "file_2.h"  
  
// some code here....  
  
// -----  
// file_1.h  
// -----  
//  
//... other declarations  
  
// -----  
// file_2.h  
// -----  
//  
//... other declarations  
  
// -----  
// file_2.c  
// -----  
#include "file_3.h"
```



```
// some code here....

// -----
// file_3.h
// -----
//... other declarations
```

The elements declared in the `file_3.h` interface are used by `FILE_2` but not by `FILE_1`. As a result, we don't need to worry about any data type conflicts (or multiple definitions) between `FILE_1` and `FILE_3`.

NO - Apparently module `FILE_1` does not use any other modules. Actually, the interfaces of all modules are imported into module 1, but this is not immediately obvious just by looking at `file1.c`. If the interface of module 4 defines data structures or variables already present in module 1, we have an error.

```
// -----
// file_1.c
// -----

#include "file_1.h"

// some code here....

// -----
// file_1.h
// -----

#include "file_2.h"

// -----
// file_2.h
// -----

#include "file_3.h"

// -----
// file_3.h
// -----
```

```
#include "file_4.h"
```

💡 A possible good practice

`#include` directives should generally be placed only in `.c` files. Headers should include other headers only when absolutely necessary.

In extreme cases — such as when multiple source files require the same large set of headers — it is acceptable to consolidate them into a single *include* file.

However, this is just **one possible best practice**. It is not uncommon to find `.h` files that include other `.h` files. In some cases, **even the implementation of single-line functions may appear in a header file**, depending on the company's coding policies.

See also

Flight Software Branch C Coding Standard - 582-2000-005

Recommended C Style and Coding Standards <https://www.doc.ic.ac.uk/lab/cplus/cstyle.html>

C Coding Standard <https://users.ece.cmu.edu/~eno/coding/CCodingStandard.html>

Brian W. Kernighan, Dennis M. Ritchie, The C Programming Language, 2^a ed

1.3 Exercise

Rewrite the `time_slot` program

Split the program in

- a **main** file
- a **timeslot** *module*

Key points:

- The client (main) and the server (module) are implemented in separate files.

- **Abstraction:** The client interacts with the server solely through its defined interface, **without needing to know its internal workings**. This design allows for modifications within the server — such as renaming structure fields or changing the internal representation of time (e.g., using separate hours and minutes instead of total minutes) — as long as the interface remains unchanged, meaning the client code does not require any updates.
- **Lack of Encapsulation:** Although the client is intended to use the provided interface, it can also choose to access the server's fields directly. This approach is risky because it bypasses any validation or checks enforced by the interface functions, and it makes the client vulnerable to breaking if the server's internal implementation (such as field names) changes.

1.4 Solution

Main:

```
#include <stdio.h>
#include "timeslot.h"

int main()
{
    TimeSlot ts;
    ts= set_h_min(1, 10);

    printf("TIME SLOT (h, m): ");
    print_h_min(ts);
    printf("TIME SLOT (m): ");
    print_min(ts);

    ts= set_min(70);
    printf("TIME SLOT (h, m): ");
    print_h_min(ts);
    printf("TIME SLOT (m): ");
    print_min(ts);

    getchar();
    return 0;
}
```

1 MODULARITY in C

Module - interface

```
#ifndef TIMESLOT_H
#define TIMESLOT_H

const int  minutesInHour;

typedef struct timeslot{
    int min;
}TimeSlot;

TimeSlot set_h_min(int h, int min);
TimeSlot set_min(int min);
void print_h_min(TimeSlot ts);
void print_min(TimeSlot ts);

#endif //TIMESLOT_H
```

Module - implementation

```
#include <stdio.h>
#include "timeslot.h"

const int  minutesInHour=60;

// VERSION 2

TimeSlot set_h_min(int h, int min){

    TimeSlot ts;
    ts.min=min+h*minutesInHour;
    return ts;
}

TimeSlot set_min(int min){
    TimeSlot ts;
```

```

    ts.min=min;
    return ts;
}

void print_h_min(TimeSlot ts){
    int h, min;
    h=ts.min/minutesInHour;
    min=ts.min-minutesInHour*h;

    printf("%d : %d\n",h, min);
    return;
}

void print_min(TimeSlot ts){
    printf("%d\n", ts.min);
    return;
}

```

1.5 Exercise

- Write a program that compute the **factorial** on an integer $n, n \leq 7$, using the function **myfactorial**.
- Split the program in
 - a **main**, that calls the function **myfactorial**, and
 - a **mymath** *module* that contains the function **myfactorial**.

This is a possible main

```

#include <stdio.h>
#include "mymath.h"

int main() {
    int n=4;
    printf("%d!= %d\n",n,myfactorial(n));
    getchar();
}

```

1 MODULARITY in C

```
    return 0;
}
```

The module **mymath** must also provide the value $\pi = 3.14159\dots$, so all client modules can use this value. *What's the right place? .c or .h?*

```
/****** mymath.c *****  
float pi=3.14159;
```

NO! - Clients module can not access the module implementation (.c).

```
/****** mymath.h *****  
float pi=3.14159;
```

NO! - If several client modules import `mymath.h`, we **define the variable more than once**.

Include guards do not solve this problem. See below.

1.6 Solution

Use the header file **to declare** the data but **not to define** the data. The header indicates to client modules **the existence of some data** or function and indicates what their type is.

```
/****** mymath.h *****  
float pi; /* declared here; defined and initialized in mymath.c */  
  
// -----  
  
/****** mymath.c *****  
float pi=3.14; //exported in mymath.h
```

You can **declare** a variable multiple times, but it can be **defined** only once.

Interface:

```
#ifndef MYMATH_H
#define MYMATH_H

float pi; /* declared here; defined and initialized in mymath.c */

int myfactorial(int n);

#endif //MYMATH_H
```

Implementation:

```
#include <stdio.h>
#include "mymath.h"

float pi=3.14; //exported in mymath.h

int myfactorial(int n){
    int nMax=7;
    int f,i;
    if (n> nMax){
        printf("Too big!");
        return -1;
    }
    if (n==0) return 1;
    else {
        f=1;
        for (i=2; i<=n; i++)
            f=f*i;
        return f;
    }
}
```

1.7 Discussion about multiple declarations

When you write

```
int a;
```

1 MODULARITY in C

in your **header file**, it is considered a **tentative definition**. If multiple source files include that header, each one gets its own tentative definition of **a**.

Some compilers (or versions/configurations) allow multiple tentative definitions to be merged into a single “common” symbol. This behavior is enabled by the `-fcommon` flag in CMake options

```
-DCMAKE_C_FLAGS="-fcommon"
```

With `-fcommon`, even though both `main.c` and `module.c` include

```
int a;
```

the linker merges them, and everything works as you expected.

In contrast, with `-fno-common` (which is the default for some newer compilers), each tentative definition is treated as a **separate definition**, the linker finds multiple conflicting definitions and raises an error.

Best Practice

Declare the variable in the header with the **extern** keyword

```
// module.h
extern int a;
```

1.8 Other details

#include <file> This form is used for system header files. It searches for a file named ‘file’ in a standard list of system directories.

#include "file" This form is used for header files of your own program. It searches for a file named ‘file’ in the directory containing the current file.

The problem of **multiple inclusion**

#include guard (macro guard or header guard) is used to avoid the problem of **multiple inclusion**.


```
/* things.h */  
#ifndef THINGS_H  
#define THINGS_H  
/* rest of include file */  
#endif
```

Suppose this file is included many times. The first time the preprocessor encounters this include file, `THINGS_H` is undefined, so the program proceeds to define `THINGS_H` and to process the rest of the file.

The next time the preprocessor encounters this file, `THINGS_H` is defined, so the preprocessor skips the rest of the file.

2 Modularity in C: Encapsulation

Abstraction

The interface is an **abstraction of the module**. The knowledge of the interface is sufficient to use the module.

- It is not **necessary** to know anything else.

Encapsulation

Encapsulation is used to **hide the internal details of a module** (its implementation). The client can only interact with the module through its interface and has no access to its inner workings. Encapsulation **restricts access** to details that are not relevant or necessary.

- It is **not possible to access** anything beyond the interface.

So far, we have focused on **abstraction**, but the proposed solution **does not implement encapsulation**.

Since the structure is defined in the interface, the `main` function also has direct access to its fields and **can manipulate them**. The client has access to the data structure **bypassing the interface functions**.

```
// module.h

typedef struct timeslot{
    int min;
}TimeSlot;

// MAIN

int main()
```

2 Modularity in C: Encapsulation

```
{  
    TimeSlot ts;  
    ts.min = -100;  
}
```

Main consequences:

- The client can bypass validation enforced by the interface functions, potentially setting inconsistent values (e.g., `ts.min = -100`)
- If the implementation changes (e.g., field names are modified), client code will break and require updates.

2.1 Encapsulation: a possible solution

Since C99, the C standard allows us to declare **objects of incomplete types** in contexts where their sizes are not required.

One common use case is defining a **pointer to an incomplete type** in a header (.h) file:

```
typedef struct MyData * MyDataPtr
```

Here, `MyDataPtr` is an alias for a **pointer to an incomplete struct type**.

Variables of type `MyDataPtr` are pointers to a struct, but the compiler does not need to know the structure's internal fields or its total memory size to handle them.

A program can use pointers like this:

```
MyDataPtr p;
```

even when the structure type is incomplete.

This works because **the memory required to store a pointer does not depend on the size of the object it points to**.

The size of a pointer is determined solely by the system's architecture (and calling conventions), not by the type or size of the pointed-to structure.

[*please read this sentence two or more times*]

Abstraction + Encapsulation

MAIN:

```
#include <stdio.h>
#include "timeslot.h"

/*
 * First Class Abstract Data Type (ADT) Pattern
 * using a pointer to an incomplete type
 */

int main() {

    TimeSlotPtr p_ts;
    p_ts = createTimeSlot();

    set_h_min(p_ts, 1, 10);

    printf("TIME SLOT (h, m): ");
    print_h_min(p_ts);
    printf("TIME SLOT (m): ");
    print_min(p_ts);

    set_min(p_ts, 70);
    printf("TIME SLOT (h, m): ");
    print_h_min(p_ts);
    printf("TIME SLOT (m): ");
    print_min(p_ts);

    destroyTimeSlot(p_ts);

    return 0;
}
```

MODULE.H:

```
#ifndef TIMESLOT_H
#define TIMESLOT_H
```

2 Modularity in C: Encapsulation

```
typedef struct timeslot *TimeSlotPtr;
//A pointer to an incomplete type
// The structure is in .c now!

/* Create a TimeSlot and return a handle to it. */
TimeSlotPtr createTimeSlot();

/* Destroy a TimeSlot.
 * All handles to it will be invalidated. */
void destroyTimeSlot(TimeSlotPtr p_ts);

void set_min(TimeSlotPtr p_ts, int min);

void set_h_min(TimeSlotPtr p_ts, int h, int min);

void print_min(TimeSlotPtr p_ts);

void print_h_min(TimeSlotPtr p_ts);

#endif //TIMESLOT_H
```

MODULE.C:

```
#include <stdlib.h>
#include <stdio.h>
#include "timeslot.h"

const int minutesInHour = 60;

/* Internal representation of MyData. */

typedef struct timeslot {
    int min;
} TimeSlot;

// nb: timeslot is a tag name; TimeSlot is an alias

// Implementation
```

```

TimeSlotPtr createTimeSlot() {
    TimeSlotPtr p_ts;
    // p_ts = malloc(sizeof (TimeSlot));
    p_ts = malloc(sizeof(*p_ts));
    return p_ts;
}

/* Destroy a TimeSlot.
 * All handles to it will be invalidated. */
void destroyTimeSlot(TimeSlotPtr p_ts) {
    free(p_ts);
}

void set_min(TimeSlotPtr p_ts, int min) {
    p_ts->min = min;
}

void set_h_min(TimeSlotPtr p_ts, int h, int min) {
    p_ts->min = (min + h * minutesInHour);
}

void print_min(TimeSlotPtr p_ts) {
    printf("%d \n", p_ts->min);
}

void print_h_min(TimeSlotPtr p_ts) {
    int h, min;
    h = p_ts->min / minutesInHour;
    min = p_ts->min - minutesInHour * h;

    printf("%d : %d\n", h, min);
}

```

In the `module.c` we define a complete structure `struct timeslot` and create an alias `TimeSlot` for it. This struct is only visible within `module.c`.

In the header file `module.h`, we declare `struct timeslot` as an **incomplete type** and define `TimeSlotPtr` as a pointer to this struct.

However, since `module.h` does not contain the full definition of `struct timeslot`, any code including this header cannot access its internal fields.

2.2 Implications

- The `main` function (or any other code including `module.h`) can declare variables of type `TimeSlotPtr`, which are simply pointers to `struct timeslot`.
- The `main` function can use these pointers, such as passing them to functions or assigning them to other pointer variables.
- However, the `main` function cannot *dereference* these pointers (i.e., it cannot use `*p = ...` or `p->...`) because it does not know the internal structure of `struct timeslot`. This is due to the fact that the full definition is hidden inside `module.c`.

By hiding the structure definition inside `module.c` and only exposing a pointer type (`TimeSlotPtr`) in `module.h`, we achieve **data encapsulation**. This ensures that external code cannot directly modify the structure's fields, enforcing better abstraction and maintainability.

2.3 Managing the Data

Since external code has access only to `TimeSlotPtr` and not the actual structure, we must provide a complete set of functions to manipulate `TimeSlot` objects. These functions should include:

- Creation functions to allocate and initialize `TimeSlot` instances.
- Getter and setter functions to access or modify the struct's data.
- Destruction functions to properly free allocated memory.

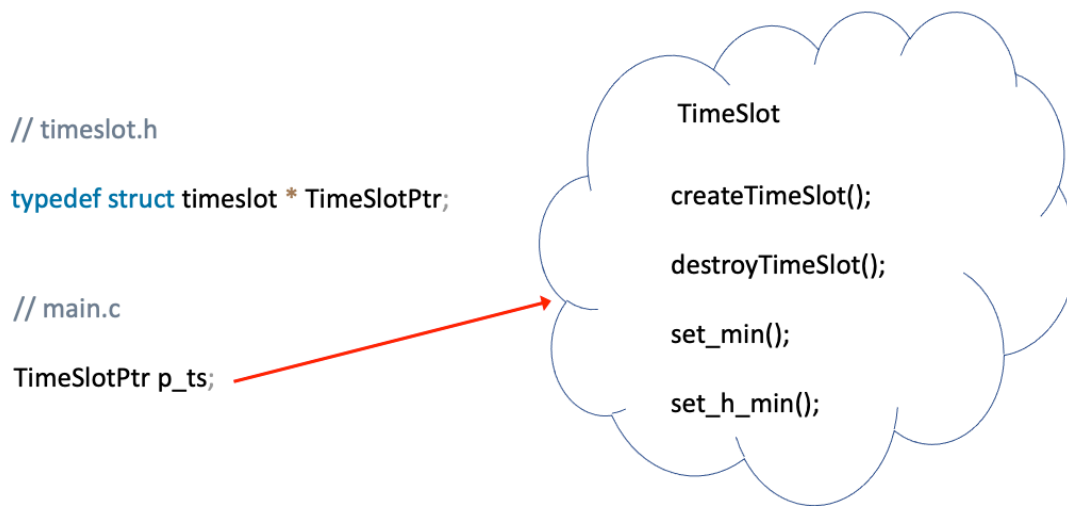
2.4 Why Must Memory Be Allocated Dynamically?

Since `main` only has a pointer (`TimeSlotPtr`), it cannot declare a complete `TimeSlot` object on the stack

```
TimeSlot ts;  
// this is not possible in MAIN
```

The only way to obtain a usable `TimeSlot` instance is to allocate it dynamically using `malloc()` (or similar functions). Similarly, we must provide a function to properly deallocate the memory when it is no longer needed to prevent memory leaks.

This approach promotes **modularity**, **encapsulation**, and better **maintainability** in C programs.



2.5 Some notes on modularity

In C, modularity is typically achieved by organizing code into a header file (`.h`) and a source file (`.c`) for each module.

The header file (`.h`) defines the interface, while the source file (`.c`) contains the implementation.

This separation improves maintainability, promotes code reuse, and enforces better abstraction.

Option 1: Exposing Data Structures in the Interface (header file)

Data structures are defined in the header file, making them directly accessible to all client modules. While this approach is convenient, it does not provide full encapsulation

Advantages:

2 Modularity in C: Encapsulation

- Convenient: Clients can declare and manipulate data structures directly.
- No need for dynamic memory allocation; structures can be allocated on the stack.

Disadvantages:

- Breaks encapsulation: Clients can access and modify the structure's fields directly, bypassing module functions.
 - Tight coupling: If the structure definition changes, all client code that directly accesses it must also be updated, leading to potential compatibility issues.
-

Option 2: Using Full Encapsulation

- The structure definition is hidden inside the .c file.
- The interface exposes only a pointer to the data structure and a set of functions to manipulate it

Advantages:

- Strong encapsulation: Clients can only interact with the data through module functions, preventing unintended modifications.
- Implementation flexibility: Internal structure changes do not affect client code, as long as the interface remains unchanged.

Disadvantages:

- Requires dynamic memory allocation, since clients only have access to a pointer and cannot allocate structures directly.
- Slight performance overhead due to indirect access via function calls.

2.6 Un esempio di memory leakage

(Sedgewick, 4.8 - First class ADT)

Consideriamo un modulo che debba gestire vettori 2d mediante incapsulamento totale del dato. Vogliamo implementare anche l'operazione **somma di vettori**

```
Vect2d * sum( Vect2d v1, Vect2d v2 );
```

2.7 Soluzione

MAIN

```
#include <stdio.h>
#include "vect2d.h"
int main() {
    VectPtr v1, v2, v3;

    v1 = create_vect(2,3);
    v2 = create_vect(8,7);
    v3 = sum_vect(v1, v2);
    print_vect(v3);

    free_vect(v1);
    free_vect(v2);
    free_vect(v3);
    return 0;
}
```

vect2d.h

```
#ifndef VECT2D_H
#define VECT2D_H

typedef struct vect_tag_name * VectPtr;
VectPtr create_vect(float x1, float x2);
VectPtr sum_vect(VectPtr v1, VectPtr v2);
void free_vect(VectPtr z);
void print_vect (VectPtr z);

#endif //VECT2D_H
```

vect2d.c

```
#include <stdlib.h>
#include <stdio.h>
#include "vect2d.h"

//typedef struct vect_tag_name * VectPtr;
typedef struct vect_tag_name{
    float x1;
    float x2;
} Vect;

VectPtr create_vect(float x1, float x2){
    VectPtr p;
    p = malloc(sizeof(*p));
```

2 Modularity in C: Encapsulation

```
    if (!p) {
        // Handle allocation error, e.g., print an error message or return NULL.
        return NULL;
    }
    p->x1=x1;
    p->x2=x2;
    return p;
};

VectPtr sum_vect(VectPtr v1, VectPtr v2){
    VectPtr p;
    float x1, x2;
    x1= v1->x1+ v2->x1;
    x2= v1->x2+ v2->x2;
    p = create_vect(x1, x2);
    return p;
}

void free_vect(VectPtr p){
    free(p);
}

void print_vect (VectPtr z){
    printf("\n ( %.02f ,  %.02f ) ", z->x1, z->x2);
};
```

2.8 Discussione

Cosa succede con le seguenti operazioni?

```
v3 = sum( v1, v2 ); //ok

v1 = sum( v1, v2 ); //memory leakage!
// non posso deallocare la memoria precedentemente gestita da v1!

// possibile soluzione (1)
void sum( Vect2d v_res, Vect2d v1, Vect2d v2 );
// semantica: v_res <- v1 + v2, e se v1== vres, devo deallocare prima

// possibile soluzione (1)
void sum( Vect2d v1, Vect2d v2 );
// semantica: v1 <- v1 + v2
```

2.9 Common errors

CORRECT:

module.h

```
// file mod1.h

#ifndef MODULE_H
#define MODULE_H

typedef struct tag_name *MyDataPtr;

MyDataPtr test();

#endif
```

module.c

```
// file module.c
#include <stdio.h>
#include <stdlib.h>
#include "module.h"

typedef struct tag_name {
    int a;
    int b;
} MyData;

MyDataPtr test(){
    MyDataPtr p= malloc( sizeof (MyData) );
    // better: malloc(sizeof *p);
    p->a=10;
    p->b=20;
    printf("\n OK! \n");
    return p;
}
```

main

```
#include <stdio.h>
#include "module.h"
int main(void) {
    MyDataPtr p;
    p= test();

    return 0;
}
```

CORRECT:

module.h

```
// file module.h

#ifndef MODULE_H
#define MODULE_H

typedef struct tag_name *MyDataPtr;

MyDataPtr test();

#endif
```

module.c

```
// file module.c
#include <stdio.h>
#include <stdlib.h>
#include "module.h"

struct tag_name {
    int a;
    int b;
};

MyDataPtr test(){
    MyDataPtr p= malloc( sizeof (struct tag_name) );
    // better: malloc(sizeof *p);
    p->a=10;
    p->b=20;
    printf("\n OK test2! \n");
    return p;
}
```

main

```
#include <stdio.h>
#include <stdlib.h>
#include "module.h"

int main(void) {
    MyDataPtr p;
    p= test();
```

```

    return 0;
}

```

WARNING:

module.h

```

// file module.h

#ifndef MODULE_H
#define MODULE_H

typedef struct MyData *MyDataPtr;

MyDataPtr test();

#endif

```

module.c

```

// file module.c
// WARNING!

#include <stdio.h>
#include <stdlib.h>
#include "module.h"

typedef struct {
    int a;
    int b;
}MyData;
// Here, MyData is an alias for a *nameless* struct.
// There is no `struct MyData` data type.
// there is no relationship between MyData and MyDataPtr

MyDataPtr test(){
    MyData * p= malloc( sizeof ( MyData) );
    // better: malloc(sizeof *p);
    p->a=10;
    p->b=20;
    printf("\n OK! \n");
    return p;
    // warning: incompatible pointer types returning 'MyData *'
    // from a function with result type 'MyDataPtr'
}

```

2 Modularity in C: Encapsulation

main

```
#include <stdio.h>
#include "module.h"
int main(void) {
    MyDataPtr p;
    p= test();

    return 0;
}
```

WRONG!:

module.h

```
// file module.h

// ERROR!

#ifndef MODULE_H
#define MODULE_H

typedef struct MyData *MyDataPtr;

MyDataPtr test();

#endif
```

module.c

```
// file module.c
// ERROR!

#include <stdio.h>
#include <stdlib.h>
#include "module.h"

typedef struct {
    int a;
    int b;
}MyData;
// Here, MyData is an alias for a *nameless* struct.
// There is no `struct MyData` data type.
// there is no relationship between MyData and MyDataPtr

MyDataPtr test(){
```



```
MyDataPtr p= malloc( sizeof ( struct MyData) );

p->a=10;
p->b=20;
printf("\n OK! \n");
return p;
}
```

main

```
#include <stdio.h>
#include "module.h"
int main(void) {
    MyDataPtr p;
    p= test();
    return 0;
}
```


3 ESERCIZI

Svolgere i seguenti esercizi implementando la modularità in C. Implementate il main nel file `main.c` e le funzioni e strutture dati richieste in almeno un modulo. Ciascun modulo deve essere composto di (almeno) un file `.h` ed un file `.c`.

Ove necessario ciascun modulo deve mettere a disposizione almeno le funzioni per leggere (getters) e scrivere (setters) i dati. In caso di allocazione dinamica la memoria deve essere correttamente deallocata dopo l'uso.

Risolvere (A) utilizzando solo il concetto di Astrazione e (B) implementando l'incapsulamento completo.

-
1. Implementare un modulo `TimeSlot` che immagazzina intervalli di tempo. La rappresentazione interna (minuti totali o minuti e ore) può cambiare senza che l'interfaccia cambi. Si implementi:
 - una funzione booleana che renda 1 se due intervalli sono uguali. Variante: 1 per intervalli quasi uguali.
 - una funzione che sommi due intervalli di tempo. Varianti: funzione `void` con due argomenti, che inserisce la somma nel primo argomento; funzione `void` con tre argomenti, il primo argomento conterrà la somma, gli altri due conterranno gli addendi; funzione che rende direttamente un `timeslot` (caso A) o un puntatore a `timeslot` (caso B)
 2. Implementare un modulo per la gestione degli angoli. La rappresentazione interna (gradi, radianti, ecc.) può cambiare senza che l'interfaccia cambi. Si implementino funzioni analoghe a quelle richieste per l'esercizio 1.
-

Esercitazione

3 ESERCIZI

- Implementare un modulo per la gestione dei numeri complessi. La rappresentazione interna (`re + img` oppure modulo e fase) può cambiare senza che l'interfaccia cambi. Si implementino funzioni analoghe a quelle richieste per l'esercizio 1. Nel caso (B), con funzione somma che rende un puntatore al valore calcolato, discutere gli effetti dell'operazione $z_3 = \text{somma}(z_1 + z_2)$ nella gestione della memoria.

Bozza di soluzione

Solo astrazione

- MAIN

```
#include <stdio.h>
#include "complex.h"

/* Note: the data structure is declared in the interface (complex.h)
 * therefore the main can directly access the structure data
 * bypassing interface functions
 */

int main() {
    Complex z1, z2, z3;

    z1= create_complex(2,3);
    printf("real part z1: %f\n", real(z1));
    printf("img part z1: %f\n\n", img(z1));

    z2= create_complex(4,5);

    printf("real part z2: %f\n", real(z2));
    printf("img part z2: %f\n\n", img(z2));

    z3 = sum_complex(z1, z2);

    printf("real part z1+z2: %f\n", real(z3));
    printf("img part z1+z2: %f\n\n", img(z3));

    printf("direct access to the complex number fields-> (1,2)\n");
    z3.re=1;
    z3.img=2;
    printf("real part z3: %f\n", real(z3));
    printf("img part z3: %f\n", img(z3));

    return 0;
}
```

- complex.h

```
#ifndef COMPLEX_H
#define COMPLEX_H

typedef struct complex_tag_name{
    float re;
    float img;
} Complex;

Complex create_complex(float re, float img);
Complex sum_complex(Complex z1, Complex z2);
float real (Complex z);
float img (Complex z);

#endif //COMPLEX_H
```

- complex.c

```
#include "complex.h"
Complex create_complex(float re, float img){
    Complex z;
    z.re=re;
    z.img= img;
    return z;
};
Complex sum_complex(Complex z1, Complex z2){
    Complex z;
    z= create_complex(z1.re +z2.re,z1.img +z2.img );
    return z;
    // return create_complex(z1.re +z2.re,z1.img +z2.img );
}

float real (Complex z){return z.re;};
float img (Complex z){return z.img;};
```

FULL encapsulation

- MAIN

```
#include <stdio.h>
#include "complex.h"
```

3 ESERCIZI

```
/* Note: the data structure is declared in the 'secret' part of the module (complex.c)
 * therefore the main CANNOT directly access the structure data
 * bypassing interface functions
 */

int main() {
    ComplexPtr z1, z2, z3;

    z1= create_complex(2,3);
    printf("real part z1: %f\n", real(z1));
    printf("img part z1: %f\n\n", img(z1));

    z2= create_complex(4,5);

    printf("real part z2: %f\n", real(z2));
    printf("img part z2: %f\n\n", img(z2));

    z3 = sum_complex(z1, z2);

    printf("real part z1+z2: %f\n", real(z3));
    printf("img part z1+z2: %f\n\n", img(z3));

    // it is not possible to directly access the complex number fields by bypassing the int

    //*z3.re=0;
    //z3->img=0;

    free_complex(z1);
    free_complex(z2);
    free_complex(z3);

    return 0;
}
```

- complex.h

```
#ifndef COMPLEX_H
#define COMPLEX_H

typedef struct complex_tag_name * ComplexPtr;
ComplexPtr create_complex(float re, float img);
ComplexPtr sum_complex(ComplexPtr z1, ComplexPtr z2);
ComplexPtr multiply(ComplexPtr z1, ComplexPtr z2);
void free_complex(ComplexPtr z);
float real (ComplexPtr z);
```

```
float img (ComplexPtr z);  
  
#endif //COMPLEX_H
```

- complex.c

```
#include "complex.h"  
#include <stdlib.h>  
struct complex_tag_name{  
    float re;  
    float img;  
};  
  
ComplexPtr create_complex(float re, float img){  
    ComplexPtr z;  
    z= malloc(sizeof(*z));  
    z->re=re;  
    z->img= img;  
    return z;  
};  
  
ComplexPtr multiply(ComplexPtr z1, ComplexPtr z2){  
    float re, img;  
    re= (z1->re * z2->re) - (z1->img * z2->img);  
    img= (z1->re * z2->img) + (z1->img * z2->re);  
    return create_complex( re, img);  
}  
  
ComplexPtr sum_complex(ComplexPtr z1, ComplexPtr z2){  
    ComplexPtr z;  
    float re = z1->re +z2->re;  
    float img = z1->img +z2->img;  
    z= create_complex( re, img);  
    return z;  
};  
  
void free_complex(ComplexPtr z){  
    free(z);  
}  
  
float real (ComplexPtr z){return z->re;};  
float img (ComplexPtr z){return z->img;};
```

3 ESERCIZI

Soluzione a memory leakage: definire una funzione `void` che non sovrascrive variabili ma, per svolgere operazioni di questo tipo

```
result <- c1 OP c2
```

deve essere chiamata in questo modo

```
OP(result,c1,c2)
```

Evitiamo memory leakage ma non abbiamo una sintassi comoda come in

```
result = OP(c1, c2)
```

```
void addComplex(ComplexPtr z_res, ComplexPtr z1, ComplexPtr z2) {
    z_res->re = z1->re + z2->real;
    z_res->img = z1->img + z2->img;
}

void subComplex(ComplexPtr z_res, ComplexPtr z1, ComplexPtr z2) {
    z_res->re = z1->re - z2->re;
    z_res->img = z1->img - z2->img;
}
```

Da notare che il [puntatore al] numero complesso che memorizza il risultato deve essere già presente ed inizializzato nel main prima del suo utilizzo

```
z3= create_complex(0,0);
addComplex(z3, z1, z2);
addComplex(z1, z1, z2);
```

-
4. Implementare un modulo per la gestione delle ore lavorate in un mese. Il mese può essere da 28, 29, 30 o 31 giorni. Il numero di giorni deve essere stabilito nella fase di 'creazione' del dato. Ad ogni giorno vanno assegnate le ore lavorate, comprese nell'intervallo tra 0 e `max_ore`. `max_ore` deve essere stabilito nella fase di 'creazione' del dato. Si implementino le funzioni per impostare, per ciascun giorno, le ore lavorate, e per ricavarne la media, il massimo, le ore totali, ecc.